

React.js Theoretical Notes

1. Introduction to React

React is an open-source JavaScript library used for building user interfaces, particularly for single-page applications. It was developed by Facebook to create fast, scalable, and interactive web applications. React follows a component-based architecture, meaning the entire UI is divided into small, reusable pieces called components. It uses a Virtual DOM to efficiently update and render only the parts of the UI that change, improving performance significantly.

2. JSX (JavaScript XML)

JSX is a syntax extension for JavaScript that allows developers to write HTML-like code inside JavaScript files. It makes UI structure easier to understand and write. Although it looks like HTML, JSX is transformed into regular JavaScript using tools like Babel before being executed in the browser. JSX must always return a single parent element, and special attributes like `className` are used instead of HTML attributes like `class`.

3. Components in React

Components are the building blocks of a React application. A component is an independent and reusable piece of code that returns UI elements. There are two types of components: Functional components and Class components. Modern React mainly uses functional components along with Hooks. Components help organize code efficiently and improve maintainability by separating concerns.

4. Props (Properties)

Props are short for properties and are used to pass data from a parent component to a child component. They allow components to be dynamic and reusable. Props are read-only, meaning a child component cannot modify the props it receives. This ensures a unidirectional data flow, which makes applications more predictable and easier to debug.

5. State in React

State is a built-in object that allows components to manage and store dynamic data. When the state of a component changes, React re-renders that component to reflect the updated data in the UI. In functional components, the `useState` Hook is used to manage state. State should never be modified directly; instead, setter functions must be used to update it.

6. React Hooks

Hooks are special functions introduced in React 16.8 that allow functional components to use state and lifecycle features. Common Hooks include `useState`, `useEffect`, `useContext`, `useRef`, `useMemo`, and `useCallback`. Hooks simplify code, reduce complexity, and eliminate the need for class components in most cases.

7. `useEffect` Hook

The `useEffect` Hook is used to perform side effects in functional components. Side effects include data fetching, subscriptions, DOM manipulation, and timers. By default, `useEffect` runs after every render, but developers can control its execution using a dependency array. Proper use of `useEffect` helps manage application behavior efficiently.

8. Forms and Controlled Components

In React, forms are usually handled using controlled components, where form data is managed by React state. The value of input fields is controlled by state variables, and changes are handled using event listeners such as `onChange`. This approach provides better control over form behavior and validation compared to traditional HTML forms.

9. Conditional Rendering

Conditional rendering in React allows components to display different UI elements based on certain conditions. Developers can use JavaScript techniques such as if-else statements, ternary operators, or logical AND operators to render content dynamically. This makes applications interactive and responsive to user actions.

10. Lists and Keys

React allows developers to render multiple elements using JavaScript's `map()` function. Each element in a list must have a unique `key` prop, which helps React identify which items have changed, been added, or removed. Keys improve performance and ensure efficient updates in the Virtual DOM.

11. React Router

React Router is a standard library used for navigation in React applications. It enables the creation of single-page applications with multiple views without reloading the page. Components such as `BrowserRouter`, `Routes`, and `Route` define navigation structure, while hooks like `useNavigate` and `useParams` provide dynamic routing capabilities.

12. Performance Optimization

React provides several techniques for optimizing performance, including memoization and lazy loading. `React.memo` prevents unnecessary re-rendering of components. Hooks like `useMemo` and `useCallback` optimize expensive calculations and function references. Lazy loading allows components to load only when needed, improving application speed and efficiency.

13. Best Practices in React

Writing clean and maintainable React code involves following best practices such as keeping components small, organizing files properly, avoiding unnecessary state, and maintaining a clear folder structure. Developers should focus on reusability, readability, and performance while building applications. Following best practices ensures scalability and easier collaboration in large projects.